

StarryOS 自举编译与内核改进成果汇报

BigLab-B 最终汇报 | 杨凯森 | 2026-05

主线：让 StarryOS 在 guest 内编译出 StarryOS 自己，过程分为两步推进。

1

首先实现单核

先让 StarryOS 在 guest 环境中稳定完成一次自举编译，形成单核可复现基线。

2

然后实现多核编译

再沿同一条链路推进多核，让 StarryOS 在 guest 内以多核方式完成自举编译。

BigLab-A：前置训练回顾

BigLab-A 主要作为进入 BigLab-B 前的 OS 基础训练；本次答辩只简要说明它给后续 StarryOS 工作提供的能力基础。

Rust 与 no_std 工程

熟悉 Rust crate 组织、无标准库环境、交叉编译、QE MU 运行和实验文档整理。

OS 基础概念

围绕内存、任务、文件系统、设备和 syscall 等模块建立基本的分层理解。

过渡到 BigLab-B

后续工作转向 TGOSKit / StarryOS：从教学小实验进入真实内核 PR 和大型 workload。

BigLab-B Task 1: tg-arceos-tutorial 基础练习

Task 1 是基于 rcore-os/tg-arceos-tutorial 的 test 分支，在自己的 fork 中完成 5 个规定的 exercise-* 基础练习。

仓库与分支

fork 上游
tg-arceos-tutorial/test，到自己的
仓库继续完成代码和总结文档。

本地对应仓库：github.com/yks23/tg-arceos-tutorial

完成的 5 个 exercise

exercise-printcolor
exercise-hashmap
exercise-altalloc
exercise-ramfs-rename
exercise-sysmap

训练到的能力

从基础输出、集合/哈希表、替代
allocator，到 RAMFS
rename、系统符号/地址映射，逐步
补齐 ArceOS 小实验能力。

- 这一部分不是 Harness，也不是 StarryOS PR；它是 BigLab-B 的基础训练任务。
- 完成后才进入 Task 2：把方法迁移到 TGOSKit / StarryOS 的真实内核改进。
- 后面的 Harness、PR、自举编译和多核测速都属于 Task 2 主线。

BigLab-B Task 2 阶段 1: Harness 初版

进入 Task 2 后，第一阶段尝试让模型生成测试并自动运行，用测试失败来反推 StarryOS 的 bug。

做法

- 围绕 syscall、busybox 小命令和文件系统行为生成测试用例。
- 模型读日志、提出怀疑点、继续补 case 或定位代码。

经验

- 能快速覆盖很多边界输入，但缺乏 workload 方向性。
- 真实 OS bug 数量不稳定，容易陷入泛化测试和低价值失败。

结论：纯测试挖掘可以作为补充，但不足以支撑大实验主线；后续改为真实 workload 驱动。

BigLab-B Task 2 阶段 1: Harness 当前闭环

Harness 后来从泛化测试挖掘，收敛到和 StarryOS dev 紧密联动：跑真实任务、读输出、修内核、补测例、提 PR。



自动同步

每天早上 9:00 拉取最新 dev，专门同步分支，减少上游快速演进带来的冲突成本。

自动修复闭环

在 StarryOS 上跑具体任务；一旦发现相关 bug，整理 root cause 与 Test Suite，随 OS 改动提 PR 到 dev。

PR 守护

定时检查 PR CI 状态；未通过则继续收集失败、修复、推送，直到达到可合并状态。

Task 2 PR: 12 个已合入 OS 修复

这些 PR 不按“改了多少文件”讲，而按 OS 行为边界拆分：进程线程、文件系统、网络 ABI、SMP 同步和调度前进性。

进程 / 线程 / futex

#692 robust futex cleanup
#693 vfork parent blocking
#878 teardown context

文件系统 / 设备

#695 rsex4 inode bitmap
#800 device full transfer
#844 tmpfs rename exec

SMP / 同步 / 调度

#842 CPU topology
#879 raw mutex handoff
#885 syscall entry snapshot
#926 SMP wakeup progress

网络 ABI

#694 IPv4-mapped IPv6 socket
保持 IPv6 用户态语义，内部复用
IPv4 backend。

验证标准

每个 PR
均包含问题根因、修复范围、测试用例和 CI 或本地复现证据。

背景 1: 进程创建、vfork 与 posix_spawn

很多用户态程序不是直接“运行一个命令”，而是通过 fork/vfork/clone 创建子进程，再 exec 成目标程序。

fork / clone

fork 会复制进程语义，clone 是 Linux 更底层的创建接口；线程、进程、vfork 都可以看成不同 flag 组合。

vfork

vfork 是一种更强同步语义：子进程 exec 或 exit 前，父进程应等待。因为这段时间父子可能共享地址空间，父进程过早运行会破坏状态。

posix_spawn / shell

shell、busybox、cargo 调 rustc 等路径会频繁创建子进程。posix_spawn 常用 vfork 风格实现，要求父子同步可靠。

- 老师问“为什么这个重要”：因为它是用户态启动其他程序的基础 ABI。
- StarryOS 自举编译里，cargo 会反复 spawn rustc/build.rs；这条路径不稳，大型 workload 就无法稳定推进。

重点 PR 1: vfork / clone 语义修复

这个 PR 修的是父进程等待条件，保证 vfork 类子进程在 exec/exit 前不会让父进程提前继续。

问题

- 旧实现把等待条件错误收窄，部分 CLONE_VFORK 场景没有等待。
- 父进程提前返回会破坏 shell、busybox、cargo 依赖的同步顺序。

修复与测例

- 修复 CLONE_VFORK 等待条件，保证父进程等待 vfork_done。
- 测例：test-vfork。
- 影响范围：BusyBox / shell / cargo 子进程创建路径。

成果：修复进程创建 ABI 语义，使 BusyBox、shell 和 cargo 子进程创建路径具备更稳定的行为基础。

背景 2: futex、robust list 与线程退出

futex 是 Linux 用户态同步的核心：平时锁在用户态完成，只有等待/唤醒时才进入内核。

futex 是什么

用户态锁先用原子指令竞争；竞争失败时用 futex syscall 让线程睡眠，释放锁时再由内核唤醒等待者。

robust list 是什么

线程持有锁时退出，用户态可能来不及释放锁。Linux 让线程登记 robust list，内核在线程退出时扫描并修复遗留锁状态。

为什么容易出错

robust list 指针来自用户态，本来就可能是坏地址；退出路径还常在复杂上下文里，不能因为清理失败再引入 panic。

- 老师问“OS 知识点”：这是用户态 pthread/mutex 与内核等待队列之间的 ABI。
- 真实 workload 里 build tool、runtime、pthread 都可能走这条路径；退出清理必须容错。

重点 PR 2: futex 与线程退出清理

这个 PR 的重点是让线程退出路径能够安全处理坏 robust-list entry 和 pending futex。

问题

- 用户态可以传坏 robust-list 指针，内核不能因此拖垮退出路径。
- pending futex 与普通 entry 混在一起，会让 pthread 退出/回收变得脆弱。

修复与测例

- 坏 entry 容错清理，pending 单独处理。
- 测例：test-futex-robust-list。
- 关联：teardown/context usercopy 修复退出路径。

成果：退出清理路径可以安全处理用户态异常输入，避免单个坏 robust-list 破坏进程回收流程。

背景 3: VFS、tmpfs、ext4 与真实应用

Linux 应用看到的是统一的文件系统接口，但内核底下可能是内存文件系统、ext4、设备节点或 proc/sysfs。

VFS

VFS

是统一抽象：open/read/write/rename/unlink/getdents 等 syscall 不直接绑定某一种磁盘格式。

tmpfs / ext4

tmpfs

主要在内存里维护目录项和文件对象；ext4 需要管理 inode、bitmap、block group 等磁盘元数据。

Cargo 压力

cargo

编译会制造大量小文件、临时文件、rename、unlink、目录扫描和设备/pipeline I/O，是检验 VFS 语义的真实压力源。

- 老师问“为什么 FS 会影响编译”：因为 target 目录几乎全是 metadata 和小文件操作。
- 这类 bug 往往不是单个命令失败，而是大量组合操作后出现目录项、inode 或短读写语义偏差。

重点 PR 3：文件系统与 VFS 正确性

这一组 PR 面向真实应用的文件系统访问模式，补齐 inode 分配、rename/exec 和设备传输语义。

#695 rsext4 inode bitmap

ext4 block group 可标记 inode bitmap 未初始化；allocator 需要识别并初始化，再继续分配 inode。

#844 tmpfs rename exec

tmpfs 的目录项和可执行文件生命周期要一致；rename/unlink 不能让后续 exec/open 看到错乱状态。

#800 device transfer

VFS 设备节点读写要遵守完整 transfer 语义，否则 busybox、构建脚本等用户态工具会拿到短读写。

成果：文件系统层面对真实应用的目录项、inode 和设备 I/O 行为更加接近 Linux 预期。

实验 4: StarryOS guest 内自举编译

实验目标是在 StarryOS userland 中运行 cargo build, 完成 StarryOS 自身构建并输出 PASS marker。



- 宿主对齐参考：同 StarryOS kernel 配置、同 target/profile, host cargo j1=85s, j8=29s。
- guest 最优：AArch64/HVF 8 核 StarryOS guest, 完整 self-build 331s。
- 判据：进入 userland、找到 rootfs 内 StarryOS ELF、打印 PASS、无 panic/trap/FATAL/error。

多核编译性能结果

通过脚本和构建参数调节，guest 完整 self-build 从 951s 降到 331s，约 3 倍，已稳定进入 400s 内。

951s

最慢 guest baseline

642s

默认 8 核 guest

331s

当前最快 guest

29s

host cargo j8

85s

host cargo j1

端到端真正收益：951s / 331s = 2.87x

guest 编译并行：422s / 341s = 1.24x

host 对齐参考：85s / 29s = 2.93x

链接/串行尾段：642s / 427s = 1.50x

注：host 与 guest 编译同一 AArch64/SMP8 StarryOS kernel；host 不进入 guest，链接项不是独立 link-only benchmark。

结论：完整 guest self-build 已完成；并行收益受 Cargo critical path、guest OS/FS/SMP 和串行尾段共同限制。

性能模型与测试细项

每个瓶颈都用接近 Cargo 行为的微基准拆开验证，而不是只给粗略归因。

$$T_{\text{build}}(N) = T_{\text{std/cache}} + T_{\text{serial}} + T_{\text{parallel}}/N + T_{\text{fs}}(N) + T_{\text{smp}}(N) + T_{\text{wait}}(N)$$

进程链路

fork/exec/wait 1000
次: 20/11/5/3s, 8 路
6.67x。模拟 cargo 反复
spawn rustc。

短任务 wave

24 个
build-script: 27/22/27/32
/32s。2
路最快，过并行会被
OS/FS/SMP 成本吃掉。

FS metadata

rsext4 fileio
jobs=8: 47.5s→14.5s。模
拟 target 小文件
create/write/rename/unlin
k。

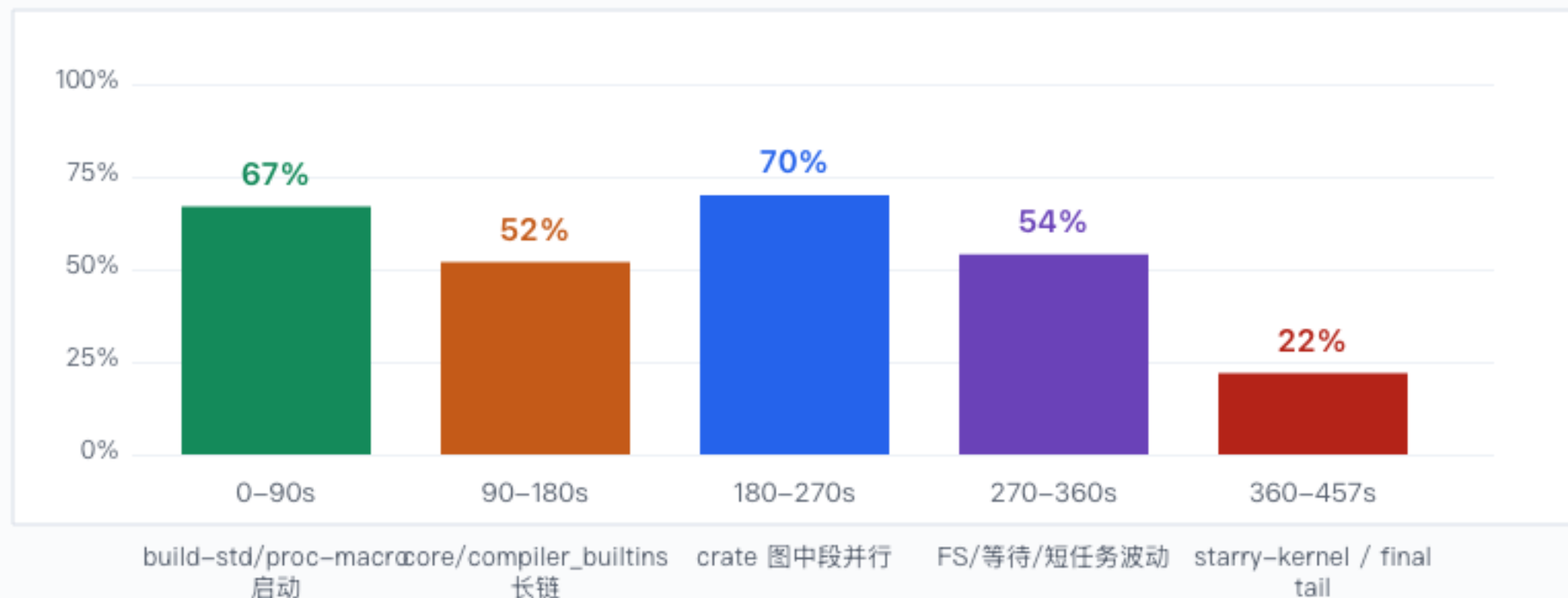
宿主/QEMU参考

host 同构建任务
85/29=2.93x; guest
jobs-only
422/341=1.24x。raw CPU
MTTCG 3.17x 不作 RISC-V
correctness。

final link timestamp: artifact tail 约 2s, starry-kernel 后段约 17–20s; 331s 版本主瓶颈不是纯链接。

SMP8 CPU 利用率与阶段分析

诊断 run 使用 GUEST_CPU_MONITOR=1 和 cargo JSON timing, 耗时 457s; 监控有开销, 但能解释 8 核为什么没有线性加速。



怎么看这张图

前中段能吃到并行, 但并不是 8 核常满; 后段掉到低利用率, 说明瓶颈转成 Cargo critical path、单个大 crate

监控暴露的问题

旧 /proc/stat aggregate 曾出现非单调, 说明 CPU 计数本身也需要内核修复; 因此这里同时结合 cargo JSON timing 与微基准解释。

结论: 8 核是有效的, 但完整 cargo 的可并行段、OS 开销和串行尾段不均衡; 下一步应优化 T_wait、T_fs、T_smp。

对齐对比：为什么 host 能 2.93x, guest 只有 1.24x

这页把同一 StarryOS kernel 构建任务的 host 参考和 guest jobs-only 放在一起，说明差距来自哪里。

85s -> 29s

host 对齐参考, 2.93x

422s -> 341s

guest jobs-only, 1.24x

951s -> 331s

guest 端到端调优, 2.87x

Host 参考

同源码、同 AArch64/SMP8
kernel config、同 release
profile; cargo/rustc
跑在宿主机。

Guest 真实承载

同 profile 只改 jobs, 但要承担
syscall、VFS、scheduler、pipe/wait 和虚拟化边界。

结论

并行空间存在; full cargo
的剩余差距来自 OS
短任务成本和串行尾段叠加。

现场讲法: host 是可并行上界; guest 是 OS 真实承载能力; 两者差距就是本实验定位和修复的对象。

成因拆解：Cargo、OS、QEMU 分别承担什么

最后把瓶颈按归属拆清楚，避免把所有慢都归因给 QEMU 或 Cargo。

观测	归因	已经做了什么
host 2.93x 但 guest jobs-only 1.24x	Cargo 图有并行空间；差距主要出在 guest 运行成本。	对齐 host/guest profile，拆分 syscall、VFS、scheduler、QEMU 边界。
fork/exec/wait 8 路 6.67x	进程创建和等待这条 OS 热路径本身可以扩展。	#842 CPU topology、#926 wakeup progress、#879 RawMutex handoff。
build-script wave 2 路最快，8 路变慢	Cargo 短任务过并行后，被 pipe/wait、文件 metadata 和锁竞争反吃。	用短基准定位 T_wait(N)、T_fs(N)、T_smp(N)，避免只等完整 build。
final link tail 约 2s	331s 版本主瓶颈不是纯链接器，而是 build graph 和 codegen 尾段。	关闭 LTO、opt0、cgu256，串行尾段 642s -> 427s。

结论：self-build 已跑通；多核不是无效，剩余瓶颈已拆成 Cargo critical path、OS 短任务成本和虚拟化边界三类

课程思考：AI 时代的 OS 实验设计

这次 BigLab-B 的经验是：AI 降低了工程实现门槛，但更凸显任务拆解、信息供给和系统理解的重要性。

1. Agent 与任务拆解

Agent

已经能处理很长的工程上下文；真正决定效率的是学生能否把目标、仓库、分支、运行模式、交付标准说清楚。

课程可以更强调：面对庞大 OS 项目，如何把模糊目标拆成可验证的小任务。

2. 更早开放 OS 实战

AI 让工程门槛显著下降，OS 课可以更多面向大一同学开放。

他们通过参与真实项目、读代码、跑测试、提 PR，会比只做小作业获得更快的工程成长。

3. 选修课的多元参与

区分必须掌握的 OS 基础和可选进阶模块。

鼓励 AI 方向同学用自己的 AI 能力完成过去单人课程里很难完成的大型系统项目，同时保持 OS 核心问题足够有趣。

结论：AI 不替代 OS 理解；它让课程更适合训练“拆解复杂系统、组织证据、完成真实工程交付”的能力。